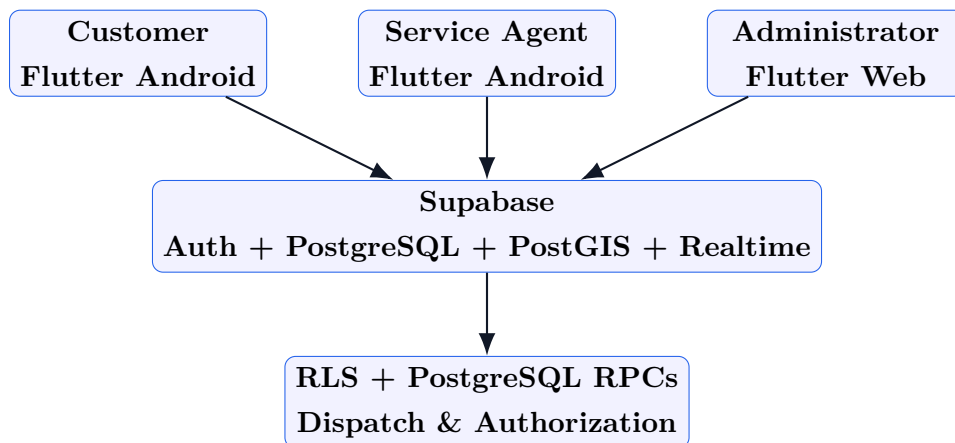


QuickServe

Architecture, Security & End-to-End System Workflow

Full-Stack Service Request and Dispatch Platform



SWASIQ Technical Assignment

Prepared by: Pranav Nimje

September 2026

Contents

1	Project Overview	3
2	Objectives	3
3	Technology Stack	4
4	High-Level Architecture	4
4.1	Architecture Responsibilities	5
5	Complete End-to-End Workflow	5
6	Customer Workflow	6
6.1	Authentication	6
6.2	Creating a Service Request	6
6.3	Customer Tracking	7
7	Service Agent Workflow	7
8	Administrator Workflow	8
9	Request Lifecycle	9
10	Smart Dispatch Architecture	9
10.1	Eligibility	9
10.2	PostGIS	10
10.3	Candidate Ranking	10
10.4	Concurrency Control	10
10.5	Offer Expiration	10
11	Realtime Architecture	11
12	Real-Time Location and Privacy	11
13	Database Architecture	12
14	Security Architecture	12
14.1	Authentication	12
14.2	Authorization	12
14.3	Row Level Security	13
14.4	Security-Definer RPCs	13

15 Administrative Authorization	14
16 Client Role Spoofing Issue and Resolution	14
17 Credential and Secret Management	15
18 Payment Architecture	15
19 Frontend Architecture	16
20 Error Handling	16
21 Testing and Verification	17
21.1 Automated Verification	17
21.2 Physical Device Verification	17
22 Web Hosting Architecture	18
23 Deployment and Setup Workflow	18
24 Key Technical Decisions	19
25 Current Limitations and Future Improvements	20
26 Conclusion	20

1 Project Overview

QuickServe is a full-stack service request and dispatch platform designed for everyday local services such as:

- AC Servicing
- Plumbing
- Electrical
- Cleaning

The platform connects three primary roles:

- **Customer:** creates service requests, schedules services, tracks request status and views payment information.
- **Service Agent:** manages availability and service radius, receives dispatch offers, accepts jobs, starts and completes services, adds completion notes and records payment information.
- **Administrator:** operates a web dashboard for monitoring requests, customers, agents, assignments and payments, and can perform authorized manual assignments.

The central architectural principle is:

The frontend handles the user experience; the backend remains the authority.

Authentication, authorization, dispatch, assignment and important state transitions are enforced by the backend rather than trusting client-side state.

2 Objectives

The main objectives of QuickServe are:

1. Provide a complete service-request lifecycle from creation to completion.
2. Separate Customer, Agent and Administrator permissions.
3. Implement backend-authoritative authorization using PostgreSQL Row Level Security.
4. Match requests with eligible service agents using geographic proximity and operational constraints.

5. Provide controlled agent offer and acceptance workflows.
6. Provide realtime operational updates.
7. Protect agent location using request-state and role-based authorization.
8. Provide secure administrator assignment operations.
9. Maintain request status history and payment records.
10. Support Android mobile workflows and a Flutter Web administration portal.

3 Technology Stack

Layer	Technology / Responsibility
Frontend	Flutter and Dart
State Management	Riverpod
Navigation	go_router
Maps	flutter_map with OpenStreetMap
Device Location	Geolocator
Authentication	Supabase Auth
Database	PostgreSQL
Spatial Processing	PostGIS
Authorization	PostgreSQL Row Level Security
Backend Logic	PostgreSQL RPC / SECURITY DEFINER functions
Realtime	Supabase Realtime
Web Hosting	Firebase Hosting
Version Control	Git and GitHub

4 High-Level Architecture

QuickServe follows a client–backend architecture.

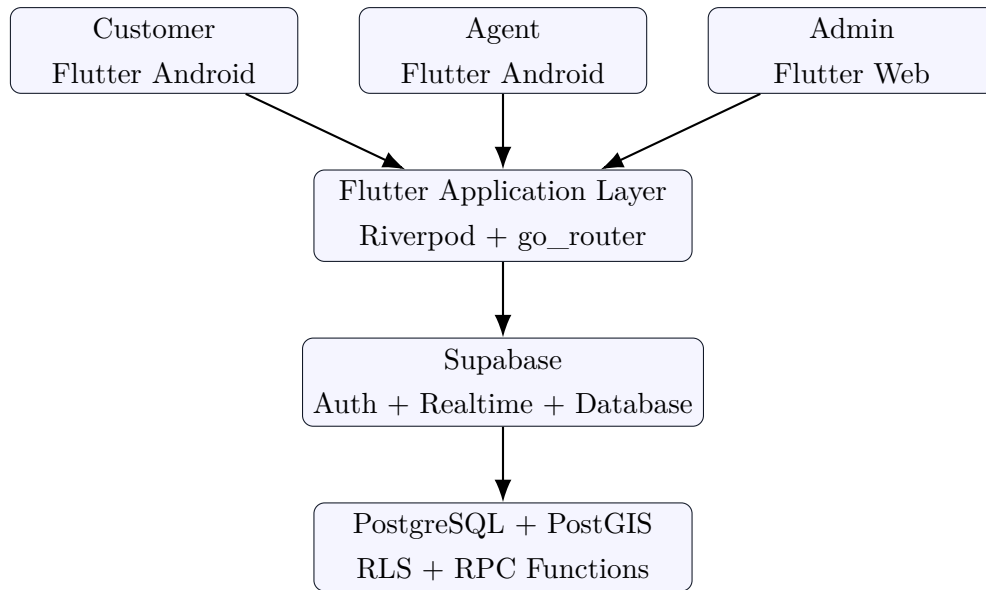


Figure 1: QuickServe High-Level Architecture

4.1 Architecture Responsibilities

- **Flutter:** UI, user interaction, local application state, navigation, device GPS and presentation.
- **Supabase Auth:** user identity and authentication sessions.
- **PostgreSQL:** persistent relational application data.
- **PostGIS:** geographic storage and spatial filtering.
- **RLS:** database-level authorization.
- **RPC Functions:** protected business operations.
- **Realtime:** operational database change streaming.
- **Firebase Hosting:** hosting of the Flutter Web application.

5 Complete End-to-End Workflow

The complete QuickServe workflow is:

Login → Create Request → Dispatch → Offer → Accept → Start → Complete

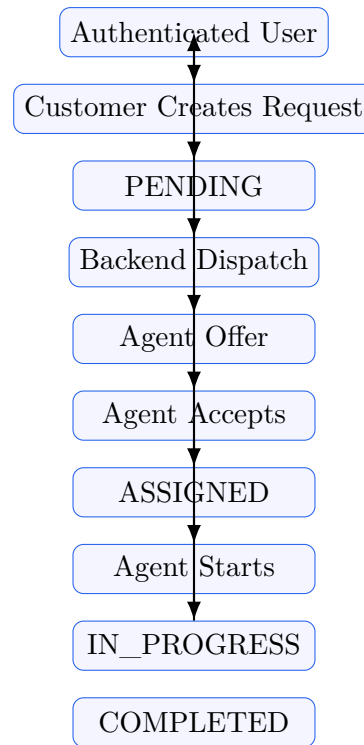


Figure 2: Primary QuickServe Workflow

6 Customer Workflow

6.1 Authentication

Public registration creates a Customer account.

The public client does not provide a mechanism for users to register themselves as Agent or Administrator.

After authentication:

1. Supabase Auth validates the credentials.
2. The authenticated session establishes the user's identity.
3. The application loads the corresponding profile.
4. The authoritative profile role determines the permitted application area.

6.2 Creating a Service Request

The customer provides:

- Service category
- Request title / description

- Service address
- Geographic coordinates
- Preferred date and time
- Priority

Every request receives a tracking identifier such as:

REQ-2026-000123

The request is then stored in PostgreSQL and the backend dispatch workflow is triggered.

6.3 Customer Tracking

The customer can view:

- Current request status
- Request history
- Assignment information when permitted
- Agent operational location when authorized
- Recorded payment information

Cancellation is restricted to eligible request states.

7 Service Agent Workflow

The Agent application provides:

1. Login and session recovery.
2. Availability control.
3. Service-radius configuration from 10–20 km.
4. Default service radius of 15 km.
5. GPS readiness.
6. Realtime dispatch offers.

7. Approximately 120-second offer response window.
8. Accept/reject operations.
9. Start service.
10. Completion notes.
11. Complete service.
12. Payment recording.
13. Job history.

An agent cannot arbitrarily assign requests to itself. Assignment eligibility is determined by backend state, location and authorization.

8 Administrator Workflow

The Administrator operates the Flutter Web portal.

Main responsibilities include:

- Dashboard and operational overview.
- Request inspection.
- Customer management views.
- Agent management views.
- Assignment management.
- Payment inspection.
- Manual assignment.

Manual assignment is performed through a protected PostgreSQL RPC rather than unrestricted client-side writes.

The principal operation is:

```
admin_assign_service_request()
```

The backend validates that the authenticated caller has the administrator role before changing assignment state.

9 Request Lifecycle

The main request lifecycle is:



Cancellation is supported for eligible states.

Terminal states such as:

COMPLETED CANCELLED

are protected from inappropriate reassignment.

Request state and assignment state are intentionally modeled separately.

10 Smart Dispatch Architecture

Dispatch runs on the backend.

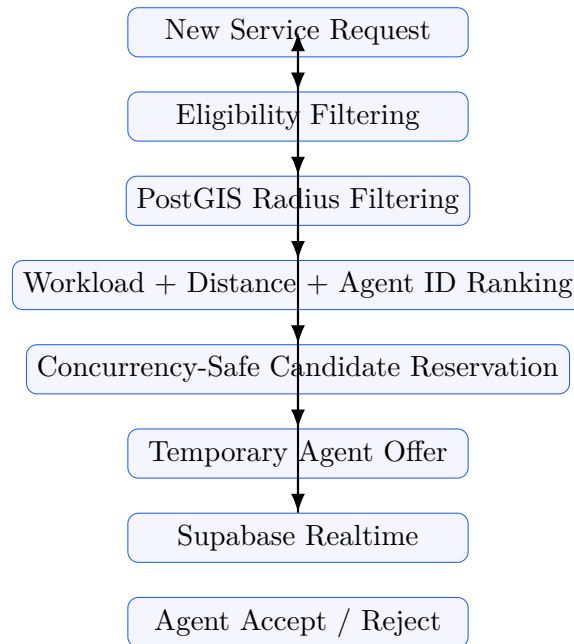


Figure 3: Server-Side Dispatch Workflow

10.1 Eligibility

The dispatch system evaluates factors including:

- Agent availability

- Agent verification
- Service capability
- Current location
- Configured service radius

The service radius is constrained to:

10 km – 20 km

with a default value of:

15 km

10.2 PostGIS

Locations are represented using PostgreSQL PostGIS geography points.

Spatial filtering uses:

ST_DWithin

This makes the backend authoritative for geographic eligibility.

10.3 Candidate Ranking

Eligible candidates are ordered using:

Workload ASC → Distance ASC → Agent ID ASC

The deterministic Agent ID tie-breaker ensures predictable ordering.

10.4 Concurrency Control

The dispatch system uses PostgreSQL row locking, including:

FOR UPDATE SKIP LOCKED

This helps prevent simultaneous dispatch transactions from incorrectly reserving the same candidate.

10.5 Offer Expiration

A dispatch offer has an approximately 120-second response window.

If the offer is rejected or expires, the dispatch process can continue toward another eligible candidate.

11 Realtime Architecture

Supabase Realtime provides operational updates.

The basic data path is:

PostgreSQL Change → Supabase Realtime → Flutter Subscription → UI Update

Examples include:

- New agent offers
- Assignment changes
- Request status changes
- Operational updates

Realtime is not treated as the only source of truth. The application can perform an initial database read and then subscribe to subsequent changes.

12 Real-Time Location and Privacy

Location access is state-dependent.

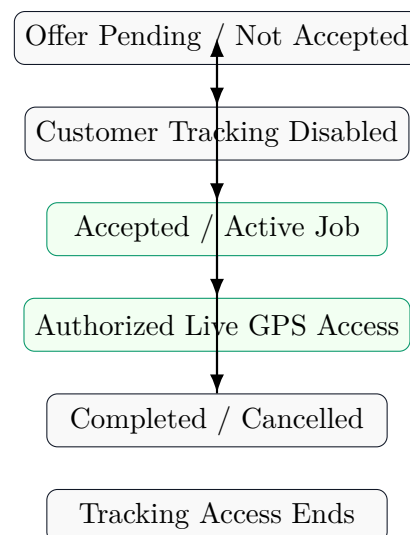


Figure 4: Location Privacy Model

The system intentionally avoids unlimited historical GPS tracking.

The current implementation uses foreground operational GPS rather than a persistent background headless location service.

13 Database Architecture

The principal database tables are:

Table	Purpose
<code>profiles</code>	User profile and authoritative role information linked to <code>auth.users</code> .
<code>agent_profiles</code>	Agent availability, service radius, verification and operational information.
<code>service_requests</code>	Customer requests, descriptions, priority, lifecycle and location.
<code>service_assignments</code>	Request-to-agent relationships, offer states and assignment timestamps.
<code>agent_locations</code>	Current operational agent GPS location using PostGIS geography.
<code>payments</code>	Payment amount, currency, method, status and timestamps.
<code>request_status_history</code>	Request state transition history.

The service request acts as the central business entity.

Conceptually:

Customer → Service Request → Assignment → Agent

with related:

History Location Payment

14 Security Architecture

Security is implemented in multiple layers.

14.1 Authentication

Supabase Auth establishes the authenticated user identity and session.

14.2 Authorization

Authorization is enforced through:

1. PostgreSQL Row Level Security.

2. Authenticated session identity.
3. Authoritative profile roles.
4. Protected PostgreSQL RPCs.

The frontend is therefore not considered a security boundary.

14.3 Row Level Security

RLS prevents unauthorized database rows from being read or modified even when a user attempts to bypass the Flutter UI.

The architectural principle is:

The UI controls the experience; RLS controls actual database access.

Examples:

- Customers access their own requests.
- Agents access operational information associated with their assignments.
- Administrators receive authorized operational access.
- Location access depends on role and request state.

14.4 Security-Definer RPCs

Sensitive backend operations are implemented through controlled PostgreSQL functions.

Examples include:

- `dispatch_service_request()`
- `accept_service_offer()`
- `reject_service_offer()`
- `expire_stale_service_offers()`
- `admin_assign_service_request()`

These functions validate backend state and authorization before performing sensitive operations.

15 Administrative Authorization

Administrative assignment is protected at the backend.

Conceptually, the RPC verifies:

```
caller_id = auth.uid();

caller_role =
    SELECT role
    FROM public.profiles
    WHERE id = caller_id;

IF caller_role != 'admin'
THEN
    reject operation;
END IF;
```

Therefore, displaying an Admin interface in Flutter does not itself grant administrative privileges.

The authenticated Supabase identity and database profile remain authoritative.

16 Client Role Spoofing Issue and Resolution

During physical-device testing, a development-only role switcher was identified.

The switcher could change the local Flutter role state without changing the actual Supabase authenticated user.

This created a dangerous mismatch:

Flutter UI = ADMIN Database Session = CUSTOMER

The backend correctly rejected the administrator RPC.

The issue was resolved by:

1. Removing the client-side role switcher.
2. Removing the role-changing method from the authentication controller.
3. Removing client-side profile-role mutation functionality.
4. Deriving application role from the authoritative profile.
5. Keeping PostgreSQL RLS and RPC authorization as the final authority.

The final role flow is:

Supabase Auth → profiles → Authoritative Role → Flutter Auth State → UI

This incident demonstrated why frontend role state must never be treated as a security boundary.

17 Credential and Secret Management

The Flutter application uses only public Supabase client configuration.

The following must never be bundled into the client:

- Supabase service-role key
- Database password
- Private API keys
- Authentication tokens
- Backend secrets

The local `.env` file is excluded from Git.

The repository contains only safe configuration placeholders through `.env.example`.

A final secret-hygiene check was performed before the final repository push.

18 Payment Architecture

QuickServe currently implements payment as operational payment recording rather than payment processing.

The payment record contains information such as:

- Amount
- Currency
- Payment method
- Payment status
- Payment timestamp
- Related request
- Customer and agent references

The current implementation does not claim to process real card or UPI transactions through an external gateway.

19 Frontend Architecture

The Flutter application follows a feature-oriented structure.

```
lib/  
|  
+-- core/  
| +-- configuration/  
| +-- routing/  
| +-- theme/  
| +-- utilities/  
|  
+-- data/  
| +-- repositories/  
| +-- services/  
|  
+-- features/  
    +-- auth/  
    +-- customer/  
    +-- agent/  
    +-- admin/  
    +-- requests/
```

The primary logical flow is:

UI → Controller / State → Repository → Supabase

This separation prevents business logic from being placed entirely inside UI widgets.

20 Error Handling

QuickServe uses centralized application error handling.

The system handles:

- Authentication failures
- Authorization failures
- Database errors
- Network failures
- Realtime failures

- Invalid request state transitions
- Assignment failures

The application converts backend failures into appropriate user-facing states rather than exposing raw internal database details.

Sensitive information is not intentionally exposed through user-facing errors.

21 Testing and Verification

QuickServe was verified through automated testing and physical-device testing.

21.1 Automated Verification

The final automated verification baseline was:

83 / 83 Tests Passing

Static analysis:

flutter analyze: 0 issues

21.2 Physical Device Verification

The release Android application was installed on:

Motorola Edge 70 Fusion
Android 16

The verified flow included:

1. Logout and session clearing.
2. Application restart.
3. Administrator authentication.
4. Authoritative role verification.
5. Fresh request creation.
6. Administrator manual assignment.
7. Protected assignment RPC execution.
8. Agent offer visibility.
9. Agent offer acceptance.
10. Request transition to assigned state.

22 Web Hosting Architecture

The Flutter Web application is hosted separately from the backend.

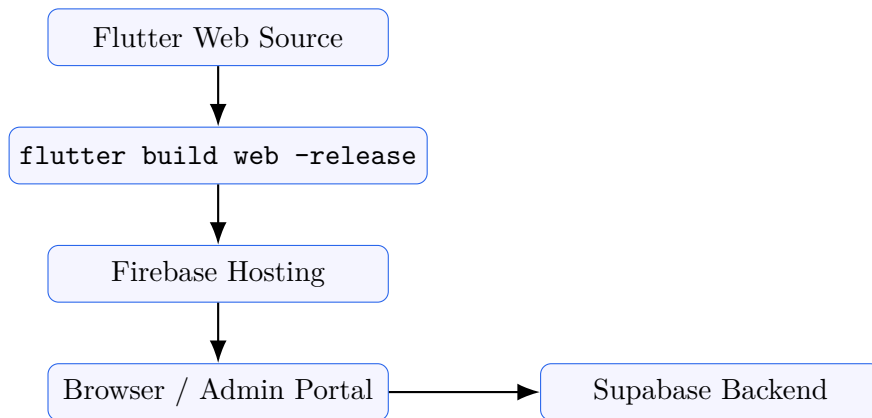


Figure 5: Web Hosting and Backend Separation

Firebase Hosting is used only to serve the Flutter Web application.
Supabase remains responsible for:

- Authentication
- PostgreSQL
- PostGIS
- RLS
- RPC functions
- Realtime

23 Deployment and Setup Workflow

A development environment follows these steps:

1. Install Flutter and Android tooling.
2. Clone the GitHub repository.
3. Run `flutter pub get`.
4. Configure public Supabase environment variables.
5. Ensure the Supabase backend is configured.

6. Run `flutter analyze`.
7. Run `flutter test`.
8. Run the Android application.
9. Run the Flutter Web portal.
10. Build the Android release APK when required.
11. Build the Web release.
12. Deploy the Web build to Firebase Hosting.

Important commands:

```
flutter pub get

flutter analyze

flutter test

flutter run

flutter run -d chrome

flutter build apk --release

flutter build web --release
```

24 Key Technical Decisions

Decision	Reason
Flutter	Shared application architecture for Android and Web.
Supabase	Provides Auth, PostgreSQL, Realtime and RLS.
PostgreSQL	Strong relational model for requests, assignments, payments and history.
PostGIS	Backend-authoritative geographic eligibility.
RLS	Database-level authorization instead of UI-only protection.

RPCs	Protected execution of sensitive business operations.
Realtime	Immediate operational updates.
Riverpod	Structured state management and dependency injection.
Foreground GPS	Operational tracking without persistent background GPS.
Firebase Hosting	Static hosting for Flutter Web while retaining Supabase as backend.

25 Current Limitations and Future Improvements

The current implementation focuses on the required core service-request workflow.

Potential future improvements include:

- Push notifications through FCM or Web Push.
- Google / Apple OAuth.
- Production payment gateway integration.
- Customer ratings and agent feedback.
- Route optimization.
- Extended analytics.
- Additional observability.
- Broader CI/CD automation.
- More extensive integration testing.

These are future improvements and should not be interpreted as current production functionality.

26 Conclusion

QuickServe implements a complete service-request workflow across Customer, Service Agent and Administrator roles.

The most important architectural property is backend authority.

Flutter provides the application experience, while:

- Supabase Auth provides identity.

- PostgreSQL stores application data.
- PostGIS performs geographic operations.
- RLS enforces database authorization.
- RPCs enforce protected business operations.
- Realtime provides operational updates.

The resulting system provides:

- Role-based workflows.
- Backend-enforced authorization.
- Spatial service-agent dispatch.
- Concurrency-aware candidate reservation.
- Temporary agent offers.
- Realtime operational updates.
- State-controlled location privacy.
- Operational payment recording.
- Administrator operational controls.
- Automated and physical-device verification.

The architecture also provides a foundation for future features without moving security-critical decisions into the untrusted client.

QuickServe

Request it. Dispatch it. Complete it.